

CH01 开发架构与框架技术概述—复习要点

1 课程概述与基本概念

1.1 课程定位与前置知识

[p.1–5]

- > **课程名称：**软件开发架构平台，基于 Java EE 平台 [p.1]
- > **前置知识 (Prerequisites)** [p.2]:
 - JavaSE —Java 标准版编程基础
 - DB/SQL —数据库与 SQL 语言
 - JDBC —Java 数据库连接技术
 - JSP/Servlet —Java Web 基础
 - HTML/CSS/XML —前端标记与样式
 - JavaScript —前端脚本语言
 - AJAX —异步请求技术
- > **课程两大方向** [p.2]:
 - **开发架构** (Development Architecture) 一面向人：系统分层 MVC、前后端分离、各种框架技术
 - **系统架构** (System Architecture) 一面向机器：数据缓存技术、服务器集群部署、服务与 REST API 设计
- > **考核方式** [p.4]: 闭卷考试 60% + 实验 40%，实验分组 4-6 人
- > **参考书籍：**《Spring in Action》[p.5]

2 MVC 开发架构

2.1 MVC 经典架构

[p.7–9]

- > 经典的 **MVC** (Model-View-Controller) 开发架构已成为 Java Web 开发的事实标准 [p.7]
- > **Model 层**进一步细分为三层 [p.8]:
 - **Service** (业务逻辑层): 完成业务逻辑处理的业务类
 - **DAO** (Data Access Object, 数据访问对象层): 实现数据持久化操作
 - **POJO** (Plain Old Java Object, 简单 Java 对象层): 仅用于表达数据的值对象
- > 完整请求流程 [p.8]: 请求 → **Servlet** (Controller) → **Service** → **DAO** → **POJO** → 数据库 → **JSP** (View) → 响应
- > 位于应用服务器和数据库服务器之间 [p.8]
- > 严格遵守 MVC 的 Web 项目有规范的文件/包结构 [p.9]

3 框架技术的由来

3.1 为什么需要框架？

[p.10–13]

- > MVC 开发中的两个核心问题 [p.10]:
 1. 在开发过程中如何约束程序员遵循 MVC 架构
 2. 在使用 MVC 架构开发过程中如何简化和规范代码
- > 对 Servlet 项目代码的分析发现 [p.13]:
 - 相同点：每个 Servlet 程序的流程基本一致，一般由三部分组成——
 1. 从客户端获取数据
 2. 调用业务逻辑进行处理
 3. 根据处理的结果响应视图
 - 不同点：具体操作的数据不一样（获取数据的个数、类型等），调用业务逻辑的方法不同，响应的 JSP 页面不同
- > 框架的核心解决思路 [p.13]：使用配置文件设定可变部分，将不可变部分用框架方式确定下来，不由程序员编写。这样既约束了 MVC 架构规范，又提高了开发效率，同时保证可维护性和可扩展性
- > 以 Servlet 为例的 Controller 代码示例 [p.11] 和 Model-持久化代码示例 [p.12] 展示了传统方式的繁琐

3.2 三类框架

[p.14]

- > 表示层框架 (Presentation Layer Framework) [p.14]:
 - 代表：Struts、Spring MVC
 - 解决 MVC 架构中的 View 层 (JSP 页面) 和 Controller 层 (Servlet 类) 的规范和简化问题
- > 持久层框架 (Persistence Layer Framework) [p.14]:
 - 代表：MyBatis、Hibernate、JPA
 - 解决 MVC 架构中 Model 层的数据访问对象代码 (DAO 包) 的规范和简化问题
- > 容器类框架 (Container Framework) [p.14]:
 - 代表：Spring、EJB
 - 解决 MVC 架构中各个组件之间的耦合问题，包括横向耦合和纵向耦合

3.3 主流框架组合演进

[p.15]

- > SSH: Spring + Struts2 + Hibernate [p.15]
- > SSM: Spring + Spring MVC + MyBatis [p.15]
- > SSM + SpringBoot: SpringBoot + Spring MVC + MyBatis（当前主流）[p.15]

3.4 框架的侵入性 (Invasiveness)

[p.24]

- > 定义 [p.24]：在软件开发过程中，由于使用第三方框架技术而导致项目自身代码的改变程度
- > 高侵入性 (High Invasiveness) [p.24]:
 - 直接继承或实现第三方框架的类或接口
 - 项目脱离框架将无法运行
 - 导致重构和单元测试效率降低，可维护性降低
 - 代表：Struts 1.x
- > 低侵入性 (Low Invasiveness) [p.24]:

- 通过反射、动态代理等语言特性，结合 **IoC** 和 **AOP** 等架构理论
 - 动态调用第三方框架的类和接口
 - 项目脱离框架依然可以运行
 - 提高可维护性和可扩展性，方便重构和单元测试
 - 代表: **Spring**、**MyBatis**
- > 软件架构设计理论中的”高内聚、低耦合”(High Cohesion, Low Coupling) 的主要目标也是降低侵入性 [p.24]

4 表示层框架详解

4.1 Struts 1 框架

[p.17–23]

- > 历史发展 [p.17]:
- 2000 年 5 月, **Craig R. McClanahan** 向 Java 社区提交 Web 框架, 这是 Struts 1 的前身
 - 2001 年 6 月, Struts 1.0 发布, 成为 ASF Jakarta 项目子项目
 - 2004 年 3 月, 成为 ASF 顶级项目
 - 2013 年 4 月, **EOL** (End of Life, 生命周期结束)
- > 基本原理 (五步配置流程) [p.18–23]:
1. 导入 Struts 1.x 对应的依赖包 [p.19]
 2. 配置 **web.xml**, 通过 **Servlet** 接管请求 [p.20]
 3. 创建 **ActionForm** 对象——封装表单数据 [p.21]
 4. 创建 **Action** 对象——需继承框架基类, 处理业务逻辑 [p.22]
 5. 完成 **struts-config.xml** 配置——定义 ActionForm 与 Action 的映射关系 [p.23]
- > 特点: 高侵入性 (Action 必须继承框架类), 配置繁琐 [p.24]

4.2 Struts 2 框架

[p.17, p.25–30]

- > 历史发展 [p.17]:
- 前身是 **WebWork** 框架, 后拆分为 WebWork 2.0 和 XWork 1.0
 - 2006 年, WebWork 2.0 和 XWork 1.0 合并改名为 Struts 2
 - Struts 2 基于 WebWork 2.3 改良, 本质上与 Struts 1.x 没有关系
 - 最新版本 2.5.26, 流行版 2.3.x
- > 核心改进思想: 约定优于配置 (Convention over Configuration) [p.25]
- > 基本原理 (五步配置流程) [p.26–30]:
1. 导入 Struts 2.x 对应的依赖包 [p.26]
 2. 配置 **web.xml**, 由 **Filter** 接管请求 (替代 Servlet 接管) [p.27]
 3. 创建业务领域对象 **POJO** 类 (普通的 Java 类, 无需继承框架类) [p.28]
 4. 创建 Action 对象——与 Java Web 容器完全解耦 (低侵入性) [p.29]
 5. 完成 **struts.xml** 配置 [p.30]
- > Struts 2 相比 Struts 1 的两大进步 [p.27, p.29]:
- Action 不需要继承框架类 → 降低侵入性
 - Filter 替代 Servlet 接管请求

4.3 Spring MVC 框架基本原理

[p.31–35]

- > 当前状态 [p.17]: Spring MVC 使用率已超过 Struts 2
- > 七步请求处理流程 [p.31]:
 1. 请求到达**前端控制器** (DispatcherServlet / Front Controller), 委托给具体控制器处理
 2. 前端控制器通过查询**处理器映射** (Handler Mapping), 找到 URL 对应的控制器
 3. 控制器处理请求, 包括处理数据、调用业务逻辑等
 4. 控制器将**模型数据** (Model) 和**逻辑视图名** (Logical View Name) 返回给前端控制器
 5. **视图解析器** (View Resolver) 将逻辑视图名匹配成具体的视图实现
 6. 视图进行模型数据和视图实现的渲染
 7. 交付模型数据, 给出 Web 响应
- > 配置要点 [p.32–35]:
 - Maven 依赖: `spring-webmvc` [p.32]
 - 配置 `web.xml` (DispatcherServlet 入口) [p.33]
 - 配置 `applicationContext.xml` (Spring 上下文) [p.34]
 - 编写 Controller [p.35]:
 - * `@Controller`: 声明该类为一个 Servlet 控制器
 - * `@GetMapping`: 注解 URL 映射, 如 `http://localhost:8080/hello`
 - * `@ResponseBody`: 直接以字符串内容进行响应

5 三大表示层框架对比

特性	Struts 1 [p.18–23]	Struts 2 [p.25–30]	Spring MVC [p.31–35]
侵入性	高 (Action 继承框架基类)	低 (POJO Action)	低 (注解驱动)
请求接管方式	Servlet	Filter	Servlet (DispatcherServlet)
核心配置文件	struts-config.xml	struts.xml	web.xml + applicationContext.xml
核心设计思想	MVC 分离	约定优于配置	注解 + REST 支持
Action 耦合度	与容器紧耦合	与容器完全解耦	与容器解耦
与 Spring 集成	需额外配置	需额外配置	原生无缝集成
当前状态	EOL (2013 年)	使用率下降	主流首选

6 Maven 项目构建管理

6.1 Maven 简介

[p.37]

- > Maven 是基于 **项目对象模型** (POM, Project Object Model) 的项目管理机制 [p.37]
- > 两大核心功能 [p.37]:

1. 通过配置文件 `pom.xml` 管理项目内部模块间和外部插件的依赖关系
2. 通过配置实现项目的自动化构建 (Build) 和部署运行

6.2 Maven 仓库体系

[p.38–39]

> 三种仓库类型 [p.38]:

- 本地仓库 (Local Repository): 本机存储已下载的依赖包
- 中央仓库 (Central Repository): Maven 官方维护, 保存大部分开源项目依赖, <https://mvnrepository.com/>
- 远程仓库/私服 (Remote Repository / Private Server): 组织或公司内部搭建的第三方仓库, 用于统一依赖版本和解决网络问题

> 依赖搜索顺序 [p.39]: 本地仓库 → 私服 → 中央仓库 → 远程仓库

> 私服配置: 在 `settings.xml` 配置文件中设置 [p.39]

> Maven 通过 `pom.xml` 识别依赖, 将项目相关依赖包下载到本地仓库 [p.39]

6.3 POM 核心概念

[p.40]

> 每个 Maven 项目有唯一的 `pom.xml` 文件 [p.40]

> 每个 `pom.xml` 有唯一表示自身的坐标 (Coordinate) [p.40]

> 坐标三要素 [p.40]:

- `groupId`: 组织或项目组标识
- `artifactId`: 模块名称
- `version`: 版本号

> 依赖声明 [p.40]:

- 通过 `<dependencies>` 子节点声明
- 每个 `<dependency>` 表示一种依赖
- 每个依赖也通过所依赖项目的坐标三要素组成

6.4 Maven 约定目录结构

[p.41]

> `src/main/java`: 项目 Java 源代码目录

> `src/main/resources`: 资源文件目录

> `src/test/java`: 测试源代码目录 [p.41]

> `src/main/web`: Web 项目相关文件 (HTML、CSS、JavaScript 等) [p.41]

> `target`: 编译结果存放目录

> `out`: 输出结果目录

> `pom.xml`: 位于项目根目录下 [p.41]

6.5 Maven 生命周期命令

[p.42]

命令	作用	说明
mvn compile	编译	将 src/main/java 中的 Java 源代码编译成 class 到 target 目录
mvn test	测试	编译并运行 src/test/java 中的测试代码
mvn clean	清理	删除 target 目录
mvn package	打包	生成打包文件，生成.jar 或.war 文件
mvn install	安装	将打包文件上传到本地仓库
mvn deploy	部署/发布	将打包文件上传到 Web 服务器或私服

6.6 主流自动化构建工具

[p.43]

- > **Maven:** 基于 XML 配置，依赖管理强大，使用最广泛
- > **Gradle:** 基于 Groovy DSL，灵活性更高，性能更好
- > **Ant:** 最早的 Java 构建工具，过程式定义，灵活但配置繁琐
- > Maven 项目的构建：可以使用 Maven 新建项目，也可将已有项目转为 Maven 项目 [p.42]

7 本章小结

[p.44]

本章作为课程导论，系统介绍了开发架构与框架技术的基本概念及发展脉络 [p.1-44]:

- > **课程概述** [p.1-5]: 课程定位、前置知识（7 项）、开发架构 vs 系统架构两大方向
- > **MVC 架构** [p.7-9]: Model 三层细分（Service/DAO/POJO），完整请求处理流程
- > **框架技术由来** [p.10-15]:
 - 两个核心问题（约束 MVC + 简化代码）
 - Servlet 程序的”相同点”与”不同点”分析
 - 框架解决方案：配置可变部分 + 固化不可变部分
 - 三类框架：表示层/持久层/容器类，各自解决的问题
 - 框架组合演进：SSH → SSM → SpringBoot+SSM
- > **框架侵入性** [p.24]: 高侵入性（Struts 1.x）vs 低侵入性（Spring/MyBatis），”高内聚低耦合”与侵入性的关系
- > **表示层框架** [p.17-35]:
 - Struts 1: Servlet 接管，五步配置，高侵入性，EOL 2013
 - Struts 2: Filter 接管，约定优于配置，POJO Action，低侵入性
 - Spring MVC: DispatcherServlet，七步请求处理流程，注解驱动，当前主流
- > **Maven** [p.37-43]:
 - POM 项目管理机制，两大核心功能
 - 仓库体系：本地 → 私服 → 中央 → 远程（搜索顺序）
 - 坐标三要素：groupId / artifactId / version
 - 约定目录结构和六条核心命令
 - 其他构建工具：Gradle、Ant

考试重点速查

高频考点：

1. MVC 架构三层细分（Service/DAO/POJO）及各层职责 [p.8]
2. 三类框架（表示层/持久层/容器类）分别解决什么问题 [p.14]
3. 框架侵入性的定义，高侵入性 vs 低侵入性的区别和代表框架 [p.24]
4. SSH → SSM → SpringBoot+SSM 的演进路径 [p.15]
5. Struts 1 五步配置流程及特点 [p.18-23]
6. Struts 2 五步配置流程，相比 Struts 1 的改进（约定优于配置、Filter 接管、POJO Action） [p.25-30]
7. Spring MVC 七步请求处理流程 [p.31]
8. Maven 坐标三要素（groupId/artifactId/version） [p.40]
9. Maven 六条生命周期命令及作用 [p.42]
10. Maven 仓库搜索顺序（本地 → 私服 → 中央 → 远程） [p.39]